Копиращ конструктор и оператор=

титуляр на курса: д-р Тодор Цонков (ttsonkov@gmail.com)

практически примери

теория

От какво ще се състои настоящата лекция?

Копиращ конструктор и оператор=. Разделна компилация. Препроцесор. Композиция и агрегация. Примери - класът Student.

Какво е копиращия конструктор?

Използва се при:
подаване по стойност, връщане по стойност, инициализация

Два механизма:

1)копиращ конструктор
2)оператор за присвояване (=)

Правилен вариант

```
class Student {
public:
    Student(const Student& other);
};
```

Извиква се само при създаване на нов обект
const – задължително
приема референция, не стойност

Кога се извиква копиращия конструктор?

```
Student s1("Ivan");
Student s2 = s1;   // copy constructor


Student s3(s1);    // copy constructor


void print(Student s) {}
print(s1); // copy constructor
```

Return Value Optimization

```
Student create() {
    Student s("Maria");
    return s; // copy ctor (или move /
RVO)
}

// Очаква се да върне копие, но RVO ще
оптимизира
// Задължително от C++ 17!

Student arr[2] = { s1, s1 };

//отново при инициализация се създава!
```

Проблем с shallow copy

Какво е плитко копие?

Плитко копие (shallow copy) е копиране на обект, при което:
се копират стойностите на членовете,
но ако обектът съдържа указатели (raw pointers), се копират само адресите, а не самите данни, към които сочат.

Лоши примери

```
class Student {
   char* name;
public:
   Student(const char* n) {
      name = new char[strlen(n)+1];
      strcpy(name, n);
   }
Student(const Student& other) {
   name = other.name; // pointer
copy
}
};
```

Компилаторът генерира плитко копие

Два обекта → един и същ name

❌ double delete

Правилен пример

```
Student(const Student& other) {
   name = new
char[strlen(other.name)+1];
   strcpy(name, other.name);
}

//Всеки студент е със собствен name
```

Какво е assignment operator=?

Student& operator=(const Student& other);
Използва се при вече съществуващи обекти

Трябва да:

1) освободи старите ресурси

2) копира новите

3) върне *this

Пример за operator=

```
Student& operator=(const Student& other)
{   if (this == &other) return *this;

    delete[] name;

    name = new char
      [strlen(other.name)+1];

    strcpy(name, other.name);

    return *this;
}
```

Rule of Three

Ако класът има:
деструктор
копиращ конструктор
оператор =

-> трябва да има и трите задължително

В следващата лекция ще се разгледа Rule of Five

Допълнение

Rule of Three важи за класове, които притежават ресурс:
динамична памет (new/delete), файл, mutex, сокет

Този ресурс трябва да се:

1) освобождава

2) копира коректно

3) присвоява коректно

Трите операции покриват трите различни момента от живота на обекта.

Разделна компилация

Какво е разделна компилация?

.h → декларации
.cpp → имплементация

Предимства:
по-бърза компилация
по-чист дизайн
независими модули

Student.h

```
#pragma once
#include <string>

class Student {
    std::string name;
public:
    Student(const std::string& n);
    std::string getName() const;
};
```

//В C++ 20 се въвеждат модулите
// за да решат проблеми - като multiple includes in headers

Student.cpp

```
#include "Student.h"
Student::Student(const std::string& n) :
name(n) {}

std::string Student::getName() const {
    return name;
}
```

Грешки при използване на header-и

```
// Student.h
class Student {
public:
    void print() const;
};

// Student.cpp
   void Student::print() {   // ❌ липсва
const
}
```

Добри практики

✔ .h = интерфейс
✔ .cpp = имплементация
✔ Минимални include-и
✔ Forward declarations
✔ Никакви .cpp includes
✔ Никакъв using namespace в header

```
-> Circular dependencies -> forward declaration
// A.h
#include "B.h"
class B;
class A {
    B b;
};

// B.h
#include "A.h"

class B {
    A a;
};
```

Какво е препроцесор?

Работи преди компилация
Основни директиви:
#include
#define
#ifdef / #ifndef

Условна компилация:

```
#ifdef DEBUG
std::cout << "Debug mode\n";
#endif
// полезна за дебъгване
```

Добри практики

```
#ifndef STUDENT_H
#define STUDENT_H
// код
#endif
};
```

```
#define PI 3.14        //лоша практика
constexpr double PI = 3.14; // добра практика
```

Обяснение на разделната компилация

Една C++ програма може да бъде разделена на множество изходни файлове (.cpp), които се компилират независимо един от друг – това се нарича разделна компилация.

Преди същинската компилация всеки изходен файл се обработва от препроцесора, който изпълнява всички директиви, започващи със символа #.

Например, при всяка среща на директивата #include препроцесорът я заменя със съдържанието на съответния хедър файл, който обикновено съдържа декларации.

В резултат на компилацията на всеки .cpp файл се получава отделен обектен файл (с разширение .obj), съдържащ машинен код.

Изпълнимият файл на програмата (с разширение .exe) се създава на следващ етап – свързване (linking) – при което линкерът обединява всички обектни файлове.

Линкерът свързва всички референции към имена (променливи, функции, класове и др.) от даден обектен файл със съответните им дефиниции, които могат да се намират в други обектни файлове.

Възможно е дадена дефиниция да липсва във всички обектни файлове. В такъв случай линкерът я търси в стандартната C++ библиотека, стандартната C библиотека, както и във всички допълнително указани от програмиста библиотеки. Ако дефиницията не бъде открита никъде, линкерът генерира грешка.

Какво е CMake?

CMake е инструмент, който генерира build файлове
(Makefile, Visual Studio, Ninja и др.) от билд конфигурация.

❓ Защо не компилираме директно с g++?
1)проектът има много .cpp файлове

2)различни платформи (Linux / Windows / macOS)

3)различни компилатори

Основна идея

➡️ Описваме проекта веднъж (CMakeLists.txt)
➡️ CMake генерира подходящ build
➡️ компилаторът върши останалото

```
project/
  ├── CMakeLists.txt
  ├── main.cpp
  ├── Student.h
  └── Student.cpp
```

Композиция и агрегация

Какво е композиция?

1) Обектът притежава другия обект

2) Животът на „частта“ е строго зависим

3) Унищожаването на цялото ⇒ унищожава и частите

4) Най-често се реализира чрез стойност (value member)

Основна идея

```
class Address {
    std::string city;
};

class Student {
    Address address;
};
```

Address се създава заедно със Student
Не може да бъде nullptr
Не се споделя с други обекти
Унищожава се автоматично (RAII)

Композиция и агрегация

Какво е агрегация?

1) Обектът НЕ притежава другия

2) Всеки има собствен жизнен цикъл

3) Често се реализира чрез:
указател, референция

4) Позволява споделяне на обекта

Основна идея

```
class Course {};

class Student {
    Course* course; // агрегация
};
```

Student не създава Course
Student не го унищожава
Course може да се споделя между много студенти
Възможно е course == nullptr

Предимства и недостатъци

Ако обектът не може да съществува без друг → композиция

Ако само го използва → агрегация

Недостатъци и рискове:

1) нужда от проверки за nullptr
2) неясен ownership
3) възможни dangling pointers

```
class Course {
public:
    void print() const {
        std::cout << "OOP course\n";
    }
};
class Student {
    Course* course;   // агрегация
public:
    Student(Course* c) : course(c)
{}
    void showCourse() const {
        // потенциален проблем
        course->print();     }
};

Student* s;

{
    Course c;
    s = new Student(&c);
} // c е унищожен тук

// undefined behavior
s->showCourse();
```

Цялостен пример на наученото досега - student.h

```
#pragma once
#include <string>
#include "Address.h"
#include "Course.h"

class Student {
    std::string name;      // RAII
    int facultyNumber;
    Address address;       // композиция
    Course* course;        // агрегация (НЕ
притежава)
public:
    Student(const std::string& n, int fn,
            const Address& a, Course* c);

    // Rule of Three
    Student(const Student& other); // copy ctor
    // copy assignment operator
    Student& operator=(const Student& other);
    ~Student(); // destructor

    const std::string& getName() const;
    const Course* getCourse() const;
};
```

Цялостен пример досега - student.cpp

```cpp
#include "Student.h"
#include <iostream>
Student::Student(const std::string& n, int fn,
                 const Address& a, Course* c)
    : name(n), facultyNumber(fn), address(a), course(c) {}

Student::Student(const Student& other)
    : name(other.name),
      facultyNumber(other.facultyNumber),
      address(other.address),
      course(other.course) {} // shallow copy
Student& Student::operator=(const Student& other) {
    if (this != &other) {
        name = other.name;
        facultyNumber = other.facultyNumber;
        address = other.address;
        course = other.course;
    }
    return *this;
}
Student::~Student() {
    // НЕ трием course → не го притежаваме
}

const std::string& Student::getName() const {
    return name;
}
const Course* Student::getCourse() const {
    return course;
}
```

Цялостен пример досега на Java

```java
public class Student {
    private final String name;
    private final int facultyNumber;
    private final Address address; // композиция
    private final Course course;   // агрегация (референция)

    public Student(String name, int fn, Address address, Course course)
{       this.name = name;
        this.facultyNumber = fn;
        this.address = new Address(address); // defensive copy
        this.course = course; // не притежаваме
    }

    // copy constructor
    public Student(Student other) {
        this.name = other.name;
        this.facultyNumber = other.facultyNumber;
        this.address = new Address(other.address);
        this.course = other.course;
    }

    public String getName() {
        return name;
    }

    public Course getCourse() {
        return course;
    }
}
```

Въпроси?

Благодаря за вниманието!

Допълнителни материали: learncpp.com - глава 14, глава 21